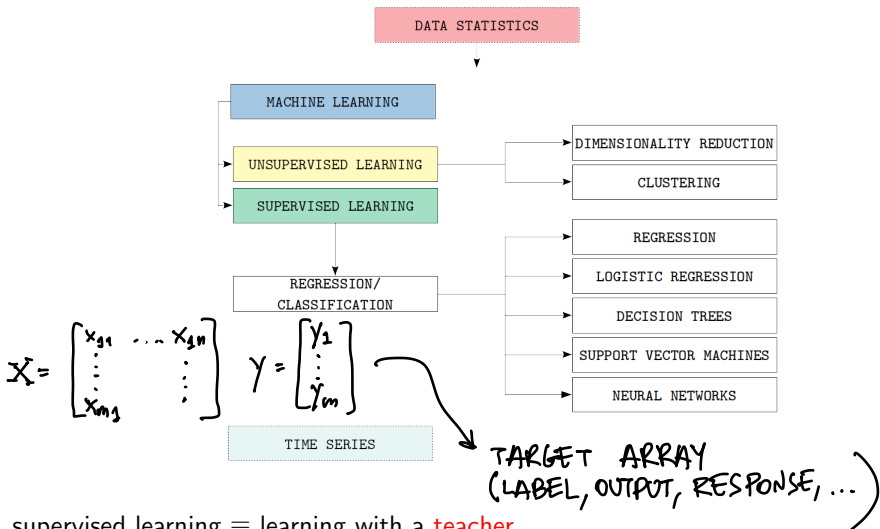


Overview

- 1 Introduction
- 2 Basic statistics
- 3 Multivariate statistics and Principal Component Analysis
- 4 Unsupervised learning: Clustering
- 5 Supervised learning**
- 6 Overview on Time Series



supervised learning $\equiv$ learning with a **teacher**

idea: observe many features-target pairs to reconstruct the function mapping features to target

supervised learning

- consider a process producing target y in correspondence to features $x \in \mathbb{R}^n$ according to an **unknown** function

✓
BUT POSTULATED

$$y = f(x) + \epsilon$$

↘ NOISE/ERROR

- supervised learning** determines a model $m(\cdot)$ approximating $f(\cdot)$ on the basis of a set of known features-target pairs denoted as **training set** (TR)

↙ HISTORICAL, AVAILABLE

INPUT - OUTPUT

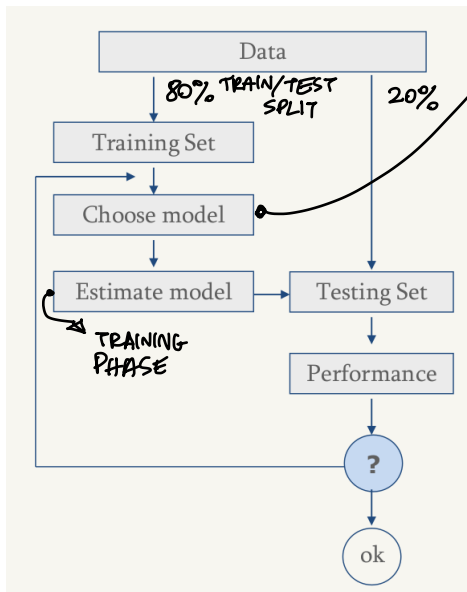
$$TR = \{(x^i, y^i) : x^i \in \mathbb{R}^n, y^i \in \mathbb{Y}, i = 1, \dots, m\}$$

- in the **training phase** the parameters of $m(\cdot)$ are **tuned** so that the model fits "as much as possible" TR
CARRIED OUT BY SOLVING AN OPTIMIZATION PROBLEM : A LOSS FUNCTION IS MINIMIZED
TRAINED
LOSS FUNCTION EXPRESSES THE DISCREPANCY BETWEEN OUTPUT OF $m(\cdot)$ AND y
- once trained the model is tested on a **testing set** (TS) of known features-target pairs **not used** in the training, and if "sufficiently good" it can be used to predict the target $\tilde{y}$ for **unseen** observations $\tilde{x}$ according to

$$\tilde{y} = m(\tilde{x})$$



supervised learning scheme



- **generalization**: capability of the trained model $m(\cdot)$ to correctly predict the target of unseen observations

NOT USED DURING THE TRAINING

- **overfitting**: when $m(\cdot)$ fits "too much" the training data and it is not able to make good predictions for unseen observations

- GOOD PERFORMANCE ON TR
- BAD PERFORMANCE ON TS

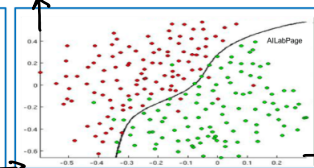
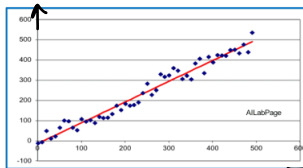
- the form of $f(\cdot)$ may be of any type
 - ▶ mathematical function
 - ▶ logic formula
 - ▶ the result of an algorithm
 - ▶ a black-box system
 - ▶ ...
- the form of $m(\cdot)$ may be of different types
 - ▶ mathematical function
 - ▶ a structured object (e.g., a tree, a neural network,...)
 - ▶ a set of rules
 - ▶ ...
- **learning tasks** categorized according to the domain of target y
 - ▶ y continuous $\Rightarrow$ **regression**
 - ▶ y discrete $\Rightarrow$ **classification** (binary or multi-class)

↳ CATEGORICAL

regression

binary classification

2-FEATURES
EXAMPLE



many issues in supervised learning...

- how to select the family of functions of model $m(\cdot)$
- **HOW TO MANIPULATE DATA** → ACCORDING TO THE TYPE OF MODEL
- how to determine **hyperparameters** of $m(\cdot)$ (model parameters which cannot be tuned during the training phase and must be set a priori) → OTHERWISE THE TRAINING PHASE WOULD BE TOO COMPLEX
- which algorithm to adopt in the training phase
- how to split training and testing sets
- how to measure the performance of the trained model METRICS TO ASSESS PERFORMANCE
- how to choose/control the model complexity (complex models fit training data well, simple models tend to better generalize)

... many alternative machine learning approaches

linear regression → REGRESSION TASK

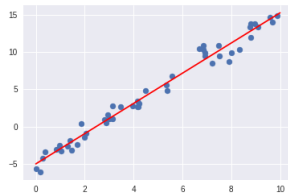
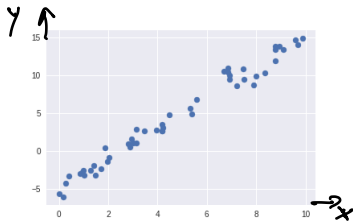
linear regression fast and interpretable → M.L. MODELS USED BY HUMANS...
*QUICKLY TRAINED
QUICK RESPONSE*

simple linear regression model (single feature)

$$y = \beta_0 + \beta_1 x \text{ (regression line)}$$

REGRESSION COEFFICIENTS

- $y \in \mathbb{R}$ (regression task)
- β_0 intercept
- β_1 slope



training a simple linear regression consists in determining $\hat{\beta}_0$ and $\hat{\beta}_1$ such that the regression line fits *AS MUCH AS POSSIBLE* better the points IN TR

training simple linear regression $\rightarrow$ SINGLE VALUE

- given a TR of pairs (x^i, y^i) with $i = 1, \dots, m$, find estimates $\hat{\beta}_0, \hat{\beta}_1$ such that

$$\hat{\beta}_0 + \hat{\beta}_1 x^i \approx y^i \quad \text{with } i = 1, \dots, m$$

APPROXIMATION $\approx$
INTERPOLATION =

- given $\hat{\beta}_0, \hat{\beta}_1$, the prediction $\hat{y}^i$ for point $x^i \in \mathbb{R}$ is

$$\hat{y}^i = \hat{\beta}_0 + \hat{\beta}_1 x^i$$

and the i -th error (residual) is computed as

$$e^i = |y^i - \hat{y}^i|$$

$\rightarrow$ PREDICTED OUTPUT

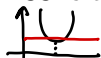
ERROR FUNCTION
 $\uparrow$

$\rightarrow$ REAL OUTPUT

- $\hat{\beta}_0, \hat{\beta}_1$ are obtained by solving the following optimization problem where a loss function called residual sum of squares (RSS) is minimized

$$\min_{\beta_0, \beta_1 \in \mathbb{R}} \text{RSS} = \sum_{i=1, m} (y^i - \underbrace{\beta_0 + \beta_1 x^i}_{\hat{y}^i})^2 = \sum_{i=1, m} (y^i - \hat{y}^i)^2 = \sum_{i=1, m} (e^i)^2$$

- RSS is STRICTLY convex $\Rightarrow$ the solution is unique:



$$\frac{\partial \text{RSS}}{\partial \beta_0} = 0$$

$$\frac{\partial \text{RSS}}{\partial \beta_1} = 0 \Rightarrow$$

$$\begin{cases} \hat{\beta}_1 = \frac{\sum_{i=1, m} (x^i - \bar{x})(y^i - \bar{y})}{\sum_{i=1, m} (x^i - \bar{x})^2} \\ \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \end{cases}$$

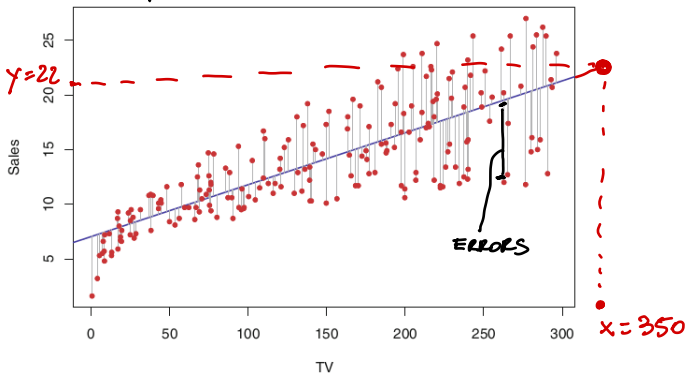
$\bar{x}$ AVERAGE OF x^i OVER TR
 $\bar{y}$ AVERAGE OF y^i OVER TR

- prediction for an unseen $\tilde{x}$: $\tilde{y} = \hat{\beta}_0 + \hat{\beta}_1 \tilde{x}$



example: Sales of a product as a function of the TV advertisings over 200 different markets

y x $m=200$



$$\hat{\beta}_0 = 7.03, \hat{\beta}_1 = 0.0475 \Rightarrow \text{Sales} = 7.03 + 0.0475 \text{ TV}$$

by spending 1000 more euro in TV advertisings we sell 47.5 more units of product!

multivariate linear regression: predictors (features) are n

**MULTIPLE SLOPE
COEFFICIENTS**

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

- given $\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2, \dots, \hat{\beta}_n$, the prediction $\hat{y}^i$ for point $x^i \in \mathbb{R}^n$ is

$$\hat{y}^i = \hat{\beta}_0 + \hat{\beta}_1 x_1^i + \hat{\beta}_2 x_2^i + \cdots + \hat{\beta}_n x_n^i$$

- $\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2, \dots, \hat{\beta}_n$ are obtained by solving

MODEL OUTPUT

$$\min_{\beta_0, \beta_1, \beta_2, \dots, \beta_n} \text{RSS} = \sum_{i=1, m} (y^i - \beta_0 - \beta_1 x_1^i - \beta_2 x_2^i - \cdots - \beta_n x_n^i)^2 =$$

REAL OUTPUT

$$= \sum_{i=1, m} (y^i - \hat{y}^i)^2 = \sum_{i=1, m} (e^i)^2$$

- the solution $\hat{\beta} \in \mathbb{R}^{n+1}$ can be computed as

$$\begin{bmatrix} \hat{\beta}_0 \\ \vdots \\ \hat{\beta}_n \end{bmatrix}$$

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

$X^{m \times (n+1)}$ = **FEATURE MATRIX**
 $y^{m \times 1}$ = **TARGET ARRAY**

- prediction for an unseen $\tilde{x}$: $\tilde{y} = \hat{\beta}_0 + \hat{\beta}_1 \tilde{x}_1 + \hat{\beta}_2 \tilde{x}_2 + \cdots + \hat{\beta}_n \tilde{x}_n$

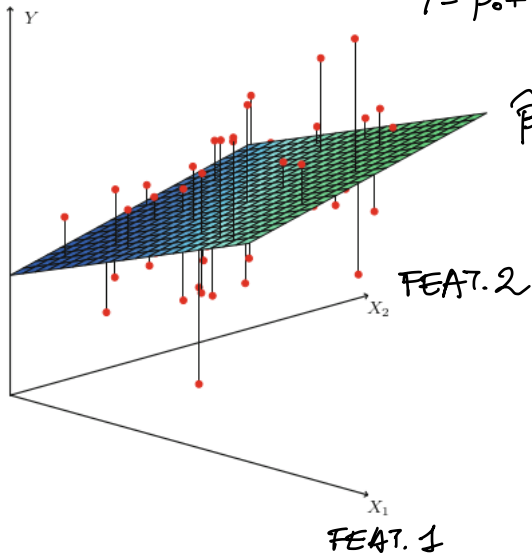
a 2-dimensional linear regression

$$n = 2$$

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

⇓ TRAINING

$$\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2$$



how to measure the regression accuracy (on the training or testing set)?

- residual sum of squares (RSS)

• IT DEPENDS ON m

• IT DEPENDS ON THE Y SCALE

• mean square error (MSE)

$$RSS = \sum_{i=1,m} (y^i - \hat{y}^i)^2$$

MODEL OUTPUT

REAL OUTPUT

THE SCALE OF RSS IS NOT THE SCALE OF Y

IT DOES NOT DEPEND ON m

$$MSE = \frac{1}{m} \sum_{i=1,m} (y^i - \hat{y}^i)^2$$

(root mean square error (RMSE) = $\sqrt{MSE}$)

THE SAME SCALE OF Y

- coefficient of determination (R^2)

INTRINSIC VARIABILITY OF THE DATASET

total sum of squares

$$TSS = \sum_{i=1,m} (y^i - \bar{y})^2$$

$\frac{1}{m} \sum_{i=1,m} y^i$

$$R^2 = 1 - \frac{RSS}{TSS} = \frac{TSS - RSS}{TSS}$$

$R^2 \in [0,1]$

ERROR OF THE MODEL

VARIABILITY THAT MODEL CANNOT EXPRESS